

1. Introducción

El proyecto consiste en el diseño e implementación de un servicio web basado en lógica declarativa. Su finalidad es permitir que un usuario envíe consultas escritas en formato Prolog por medio de una API REST desarrollada con Node.js y Express.

El sistema funciona como un motor de inferencia simbólica. Esto significa que no solamente responde datos directos, sino que también puede obtener conclusiones a partir de hechos y reglas definidos en una base de conocimiento. Por ejemplo, si un contrato tiene pago tardío y además es de alto riesgo, el sistema puede inferir que se le debe aplicar una penalización.

La idea principal del proyecto es demostrar la integración de distintos paradigmas de programación dentro de una misma solución. En este caso se combinan la programación lógica con Prolog, la programación funcional con JavaScript y la programación asíncrona propia del modelo de ejecución de Node.js.

El repositorio del proyecto se encuentra en GitHub y contiene el código fuente, la base de conocimiento y la documentación necesaria para ejecutar el sistema de forma local:

<https://github.com/MateoAlejandroCaamalTencle/motor-inferencia-logica.git>

2. Explicación de paradigmas de programación usados

2.1 Programación lógica

La programación lógica se utiliza mediante Prolog. En este paradigma se definen hechos y reglas, y el motor se encarga de determinar si una consulta se cumple o no. En lugar de escribir todos los pasos del procedimiento, se declaran relaciones lógicas y condiciones.

En la base de conocimiento del proyecto se registran hechos como contratos, pagos tardíos, clientes VIP, contratos activos, contratos de alto riesgo e historial de fraude. A partir de esos hechos se construyen reglas como `penalty_applicable`, `needs_review`, `priority_client`, `investigation_required` y `suspend_contract`.

Un ejemplo de regla utilizada en el sistema es:

```
penalty_applicable(X) :-
    contract(X),
    late_payment(X),
    high_risk(X).
```

Esta regla indica que una penalización aplica cuando existe un contrato, tiene pago tardío y además está marcado como de alto riesgo. Si las tres condiciones se cumplen para un mismo contrato, el motor puede responder que la consulta es verdadera.

2.2 Programación funcional

La programación funcional aparece en el tratamiento de los datos dentro del servidor. Antes de ejecutar una consulta, el sistema transforma la entrada recibida del cliente para dejarla en un formato más controlado.

En el código se observa la normalización de la consulta mediante funciones encadenadas:

```
const normalizedQuery = query
    .trim()
    .toLowerCase();
```

Esta parte toma la consulta enviada por el usuario, elimina espacios innecesarios al inicio y al final, y la convierte a minúsculas. También se separa la responsabilidad del motor de Prolog en una función llamada `runQuery`, lo cual ayuda a que el servidor Express no tenga toda la lógica mezclada en un solo bloque de código.

2.3 Programación asíncrona

La programación asíncrona se utiliza porque el servidor debe leer archivos y atender solicitudes HTTP sin bloquear la ejecución completa del sistema. Para esto se emplean `async/await` y `Promises`.

La función `runQuery` lee la base de conocimiento desde el archivo `knowledge.pl` usando `fs.readFile` de manera asíncrona. Después crea una sesión de Tau Prolog, carga el conocimiento y ejecuta la consulta recibida mediante una `Promise`.

```
const knowledgeBase = await fs.readFile("./src/knowledge.pl", "utf8");
return new Promise((resolve, reject) => { /* sesión Tau Prolog */ });
```

3. Arquitectura del sistema

La arquitectura del sistema es sencilla y está compuesta por un módulo principal que integra el servidor HTTP, el motor de inferencia y la base de conocimiento. La API recibe la consulta, la prepara y la envía al motor lógico para que este evalúe las reglas declaradas en Prolog.

Componente	Función dentro del sistema
Cliente	Envía una consulta Prolog por medio de una solicitud HTTP.
Servidor Express	Expone el endpoint /query y recibe la consulta del usuario.
Capa de normalización	Procesa la entrada antes de enviarla al motor de inferencia.
Motor Tau Prolog	Crea una sesión Prolog, consulta la base de conocimiento y obtiene una respuesta.
Base de conocimiento	Archivo knowledge.pl que contiene hechos y reglas declarativas.
Respuesta JSON	Devuelve al cliente el resultado de la inferencia.

El flujo general de ejecución puede resumirse de la siguiente forma:

1. El cliente envía una consulta lógica al endpoint POST /query.
2. El servidor Express recibe el cuerpo de la solicitud en formato JSON.
3. La consulta se normaliza antes de ser procesada.
4. El motor de Prolog carga la base de conocimiento desde el archivo knowledge.pl.
5. Tau Prolog evalúa la consulta usando los hechos y reglas declarados.
6. El servidor devuelve el resultado al cliente en formato JSON.

De forma general, la relación entre los elementos del sistema es la siguiente:

```

Cliente HTTP
  -> Servidor Express
    -> Endpoint POST /query
      -> Función runQuery
        -> Tau Prolog
          -> Base de conocimiento knowledge.pl
            <- Resultado de inferencia
              <- Respuesta JSON

```

4. Ejemplo de ejecución

Para ejecutar el proyecto de manera local, primero se debe clonar el repositorio de GitHub:

```
git clone https://github.com/MateoAlejandroCaamalTencle/motor-inferencia-logica.git
```

Después se ingresa a la carpeta del proyecto y se instalan las dependencias:

```
cd motor-inferencia-logica
npm install
```

Finalmente, se ejecuta el servidor:

```
npm start
```

Si el script start no está configurado, también se puede ejecutar directamente el archivo principal:

```
node src/server.js
```

Cuando el servidor inicia correctamente, debe mostrarse un mensaje similar al siguiente:

```
Servidor corriendo en puerto 3000
```

El sistema expone el endpoint POST /query, el cual recibe una consulta en formato JSON.

Un ejemplo de consulta válida es:

```
{
  "query": "penalty_applicable(contract1)."
}
```

Una respuesta esperada sería:

```
{
  "success": true,
  "query": "penalty_applicable(contract1).",
  "result": "true."
}
```

El resultado es verdadero porque contract1 aparece registrado como contrato, tiene pago tardío y también está marcado como contrato de alto riesgo. Por lo tanto, cumple con las condiciones de la regla penalty_applicable.

Otro ejemplo de consulta es:

```
{
  "query": "priority_client(contract2)."
}
```

En este caso, la respuesta esperada también es verdadera porque contract2 pertenece a un cliente VIP y se encuentra como contrato activo dentro de la base de conocimiento.

Un ejemplo donde la respuesta esperada es falsa sería:

```
{
  "query": "suspend_contract(contract4)."
}
```

Aunque contract4 es de alto riesgo y tiene historial de fraude, no tiene registrado un pago tardío. Por eso no cumple todas las condiciones necesarias para suspender el contrato.

5. Conclusiones y extensiones

El proyecto cumple con el objetivo de integrar distintos paradigmas de programación en una aplicación funcional. La parte lógica se concentra en Prolog, donde se definen reglas y hechos; la parte web se desarrolla con Node.js y Express; y la comunicación entre ambos se maneja de manera asíncrona usando JavaScript.

Una ventaja importante de este enfoque es que la base de conocimiento puede modificarse sin cambiar por completo el servidor. Si se necesitan nuevas reglas, basta con agregarlas al archivo Prolog y posteriormente consultarlas desde el endpoint correspondiente.

El sistema también muestra cómo un motor de inferencia puede exponerse como servicio. Esto permite que otros clientes, aplicaciones o interfaces puedan enviar consultas y obtener resultados sin ejecutar Prolog directamente.

Como posibles extensiones futuras se proponen las siguientes mejoras:

- Agregar una interfaz web sencilla para que el usuario pueda escribir consultas sin usar herramientas externas.
- Permitir consultas con variables y devolver múltiples respuestas cuando Prolog encuentre más de una solución.
- Mejorar la validación de entradas para evitar consultas mal formadas.
- Cargar la base de conocimiento una sola vez al iniciar el servidor para mejorar el rendimiento.
- Agregar pruebas unitarias para validar reglas importantes como penalizaciones, revisiones e investigaciones.
- Documentar el endpoint con ejemplos usando Postman o Swagger.
- Agregar autenticación básica si el servicio se usa en un contexto con información sensible.
- Permitir que la base de conocimiento se divida en varios archivos para organizar mejor los hechos y reglas.

En conclusión, el proyecto demuestra de forma clara cómo una API REST puede funcionar como intermediaria entre un cliente y un motor de inferencia declarativo. Esto permite construir soluciones donde la lógica del negocio se expresa mediante reglas, haciendo que el sistema sea más fácil de entender, extender y adaptar a nuevos escenarios.